

Homework 04 T-tests, Sampling Distributions, and the Bootstrap

Due by 11:59pm, Friday 10.3.25

S&DS 2300/5300/ENV 757

(1) Practice with Loops. (10 points) For this problem, use loops even if you could do the task without them.

(1.1) A Tribonacci sequence is a series of integers where each number after the third number is found by adding together the three integers before it. Starting with 0, 1, 1, the sequence goes:

0, 1, 1, 2, 4, 7, ...

Write a loop that fills a vector called `myTrib` with this sequence going up to a total length of 21 numbers. Display the final value of `myTrib` AND write a line of code that calculates how many digits are in the final number in `myTrib`.

(1.2) Here is the link to the World Bank data from 2024:

https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv
(https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv)

Read the data into a dataframe called `wb`. Write a loop to fill a vector called `compVals` having length equal to the number of columns in the World Bank data frame. The *i*-th entry in `compVals` should be a number (≥ 0) equal to the total number of non-missing values in the *i*-th column of World Bank data frame. Make a histogram of `compVals` and label as appropriate. In addition, make a histogram that displays the units on the horizontal axis in terms of percentages (not proportions) for each variable (rather than totals). Write a sentence or two about what these plots tell you about the dataset.

(For full credit, use only one for-loop to do part 1.2)

```
# 1.1
myTrib <- numeric(21)
myTrib[1:3] <- c(0, 1, 1)
for (i in 4:21){
  myTrib[i] <- myTrib[i - 1] + myTrib[i - 2] + myTrib[i - 3]
}

myTrib[21]
```

```
## [1] 66012
```

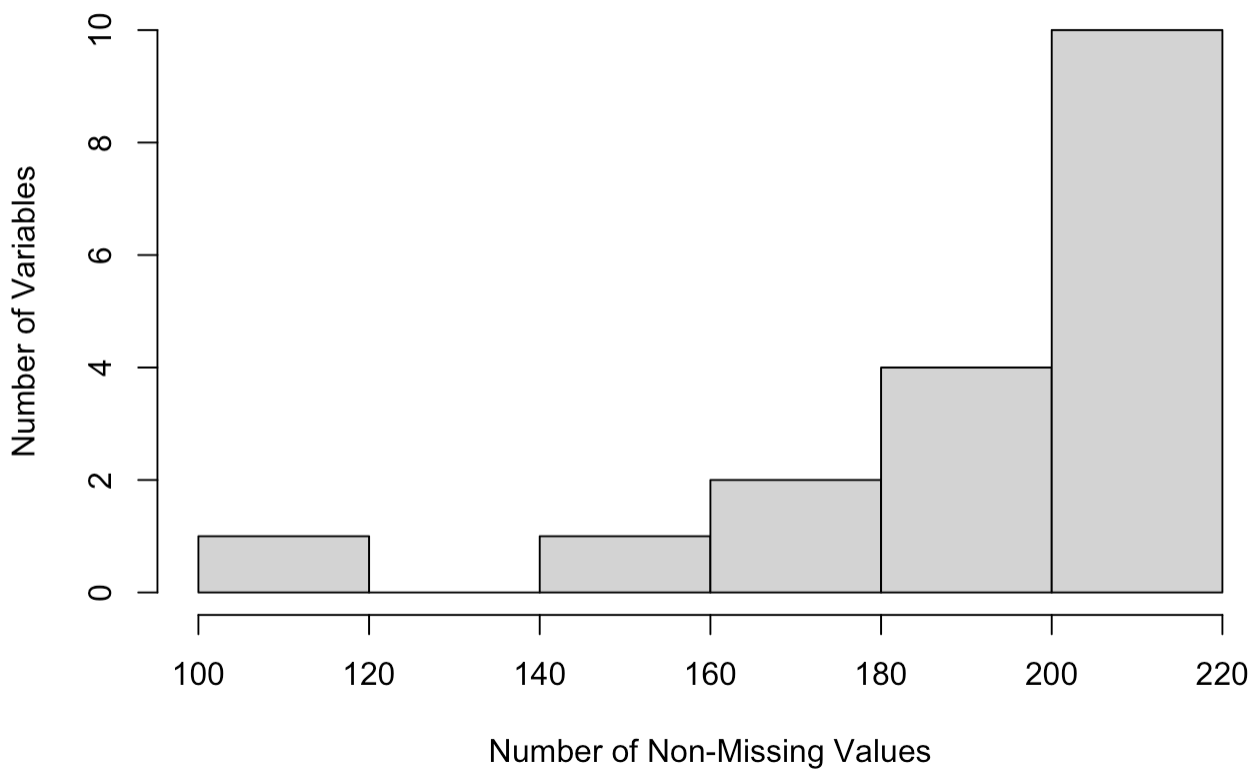
```
nchar(as.character(myTrib[21]))
```

```
## [1] 5
```

```
# 1.2
wb <- read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv")
compVals <- numeric(ncol(wb))
for (i in seq_along(compVals)){
  compVals[i] <- sum(!is.na(wb[, i]))
}

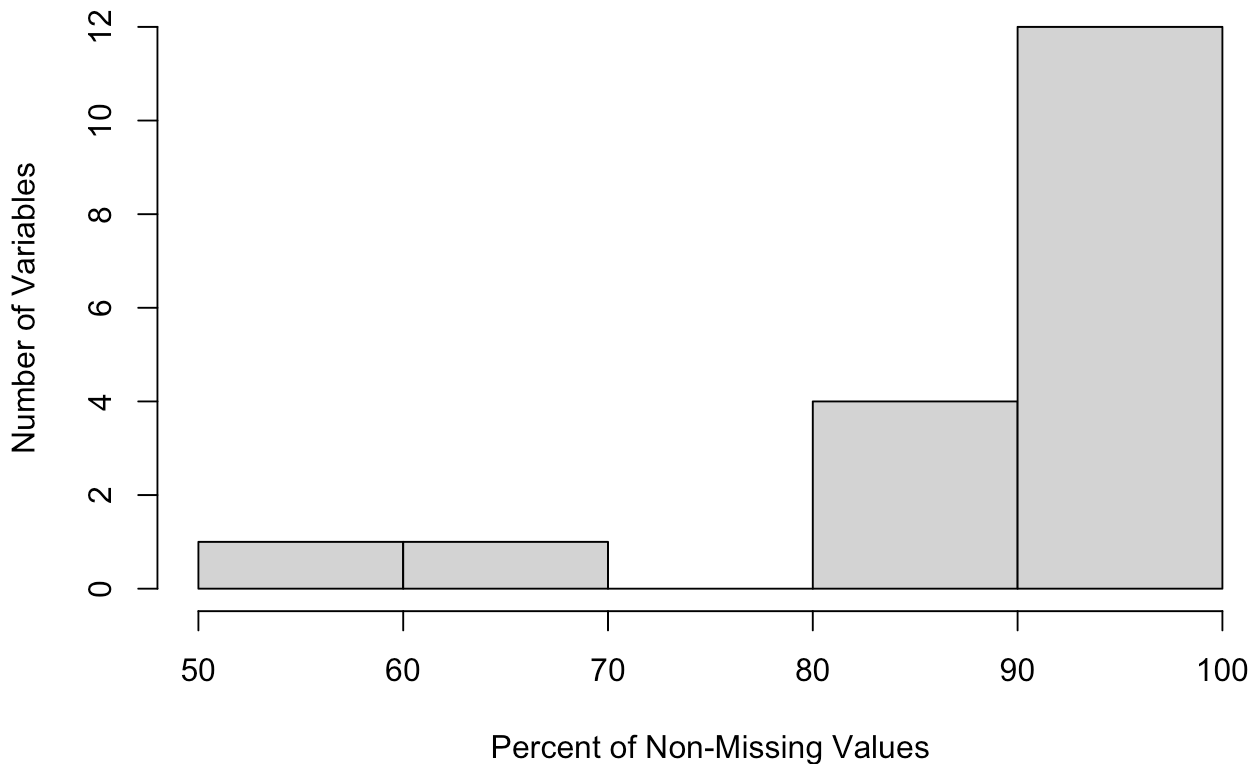
hist(compVals,
      main = "Counts of Non-Missing Values by Variable",
      xlab = "Number of Non-Missing Values",
      ylab = "Number of Variables")
```

Counts of Non-Missing Values by Variable



```
compPercents <- compVals / nrow(wb) * 100
hist(compPercents,
      main = "Percents of Non-Missing Values by Variable",
      xlab = "Percent of Non-Missing Values",
      ylab = "Number of Variables")
```

Percents of Non-Missing Values by Variable



Based on the histograms, we can tell that there are around 220 rows and that most of the variables are mostly filled. We can tell because a lot of the variables fall under 80-100% of “Percent of Non-missing Values” and the counts version of the histogram have most of the variables falling in 180-220 “Number of Non-Missing Values”.

(2) Simulations with the Exponential Distribution (40 points).

For this problem, we’ll investigate the sampling distributional characteristics of three statistics. In particular, suppose we take a sample of size 20 from an exponential distribution. We can use the CLT to say something about how far the sample mean is likely to be from the true mean, but how far are the sample median or the sample variance likely to be from the true values in an exponential distribution where we take a sample of size 20? Also, what do we expect the distribution of these statistics to look like?

(2.1) (10 points) First, let’s get a quick sense of what an exponential distribution looks like where the mean is 4. By the way, it’s handy to know that for an exponential distribution with mean 4, the variance is 16 and the median is $4 \cdot \ln(2)$. You can read about the exponential distribution [HERE \(https://en.wikipedia.org/wiki/Exponential_distribution\)](https://en.wikipedia.org/wiki/Exponential_distribution).

The code below gives a quick plot of this distribution. Your job is to succinctly answer what each part of the code does. You’ll probably need to get help on the function `dexp()`, `seq()` and on a few of the graphics parameters in `par()`.

Then, add additional code that repeats the original code but uses the option `by = 4` in the `seq()` function. Why did I choose `by = .1` in the original code?

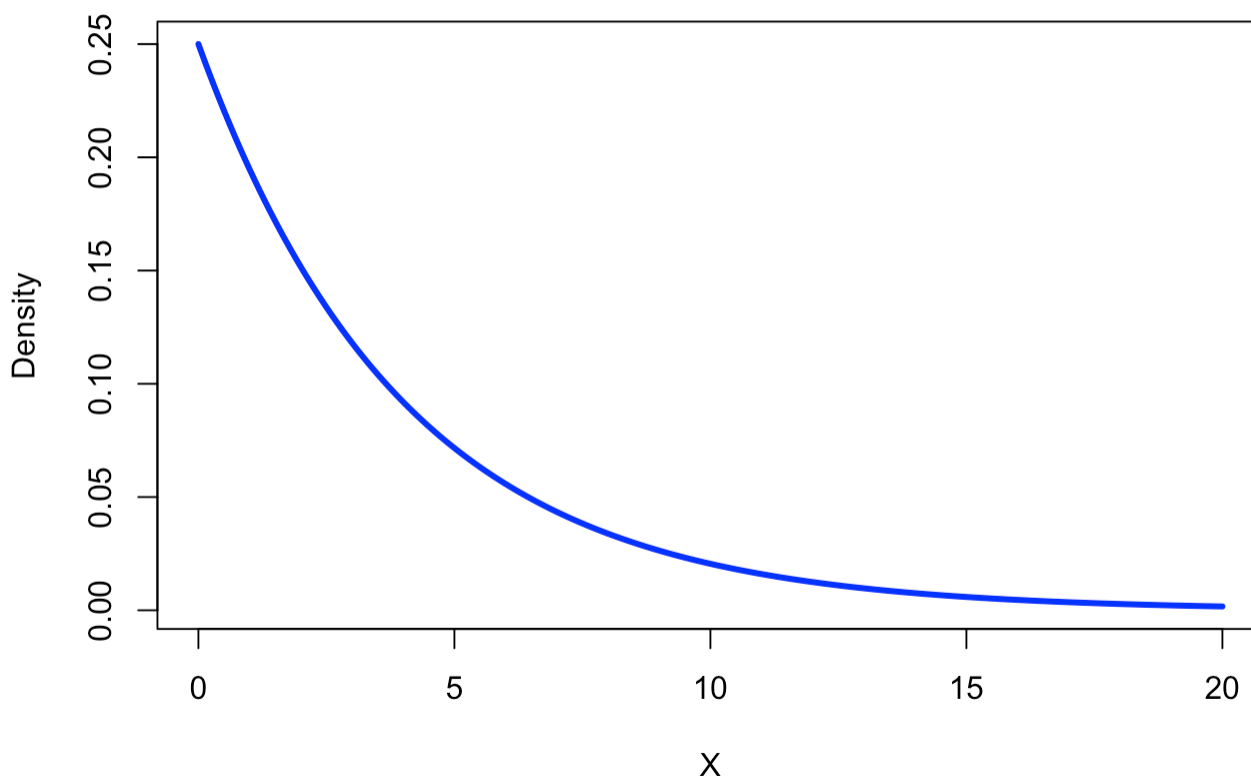
Finally, modify the code to produce a plot that shows the shape of exponential distributions with means of 1, 2, and 4 all on the same plot using different colors and line types for each distribution. Be sure to add a legend to your plot.

```
#Get exponential probabilities - note that rate = .5 gives us mean of 2 (mean is 1/rate)
probs <- dexp(seq(0, 20, by = .1), rate = .25)
```

#dexp: dexp() gives the probability density function of the exponential distribution, calculating the probabilities of the values of the first parameter (seq(0, 20, by = .1))
#rate: the rate parameter is the reciprocal of the mean of the distribution, and it dictates how frequently an event occurs
#by: by determines the increment size from the range determined by the other parameters in seq (here we would have values 0-20 incremented by 0.1)

```
#Plot sampling distribution
plot(seq(0, 20, by = .1), probs,
     type = 'l',
     lwd = 3,
     col = "blue",
     main = "Probability Distribution Function for Exponential Dist with Mean = 4",
     xlab = "X",
     ylab = "Density")
```

Probability Distribution Function for Exponential Dist with Mean = 4

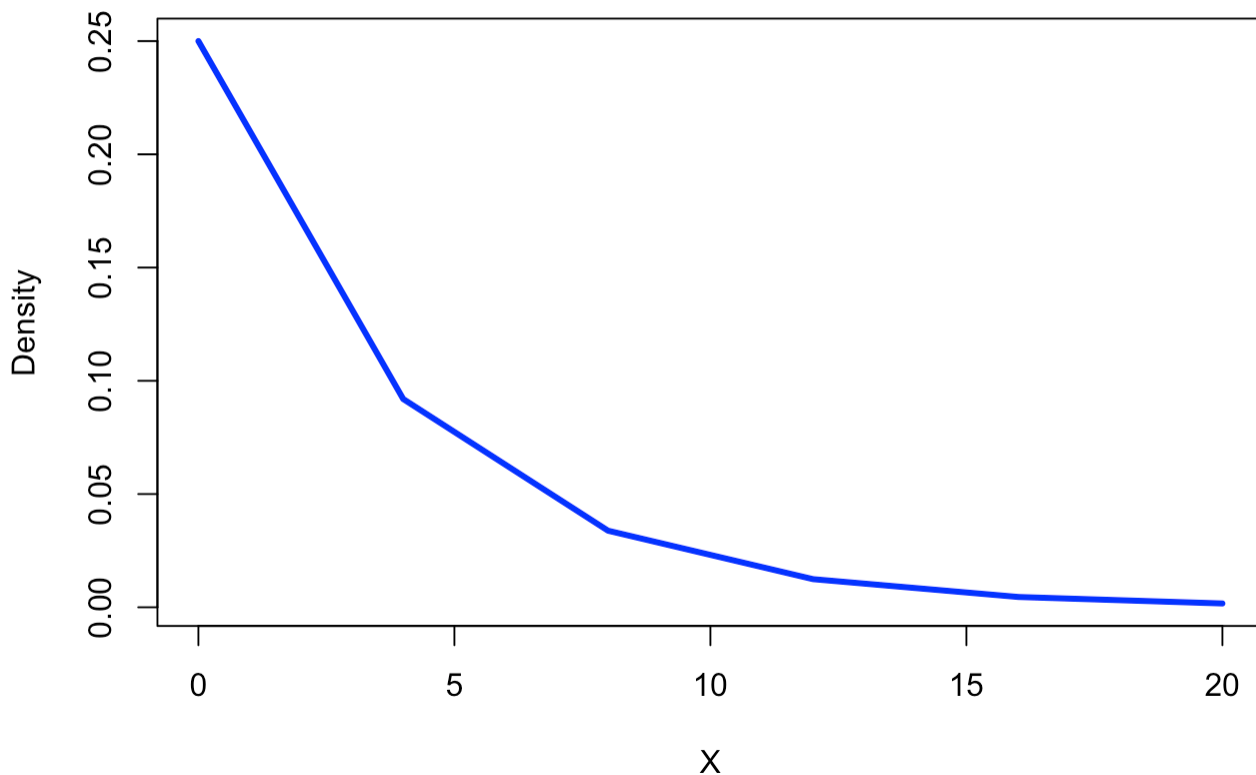


```
#type = 'l': this parameter adds lines between the data points
#lwd: lwd determines line width)
```

```
#Add code below based on instructions
```

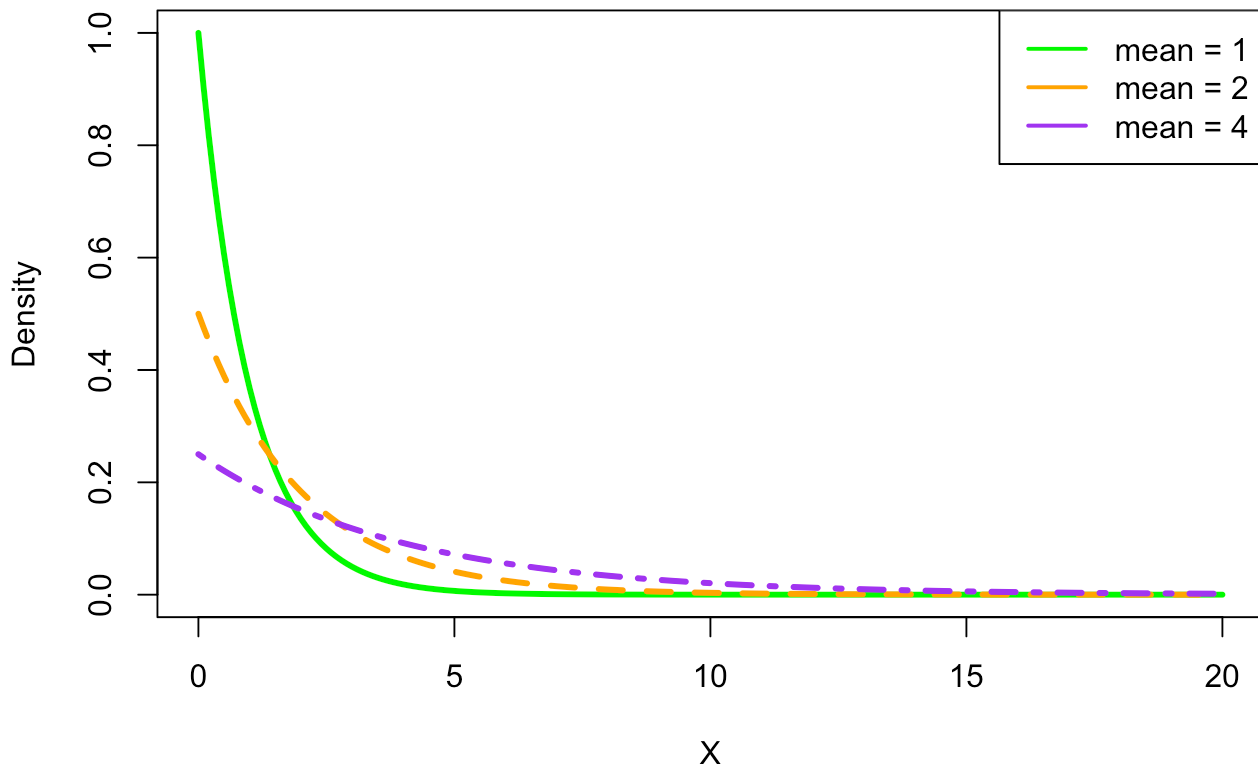
```
probsByFour <- dexp(seq(0, 20, by = 4), rate = .25)
plot(seq(0, 20, by = 4), probsByFour,
     type = 'l',
     lwd = 3,
     col = "blue",
     main = "Probability Distribution Function for Exponential Dist with Mean = 4",
     xlab = "X",
     ylab = "Density")
```

Probability Distribution Function for Exponential Dist with Mean = 4



```
probsMeanOne <- dexp(seq(0, 20, by = .1), rate = 1)
probsMeanTwo <- dexp(seq(0, 20, by = .1), rate = .5)
probsMeanFour <- dexp(seq(0, 20, by = .1), rate = .25)
plot(seq(0, 20, by = .1), probsMeanOne,
     type = 'l',
     lwd = 3,
     lty = 1,
     col = "green",
     main = "Probability Distribution Function for Exponential Dists with Means = 1, 2, 4",
     xlab = "X",
     ylab = "Density")
lines(seq(0, 20, by = .1), probsMeanTwo, lwd = 3, lty = 2, col = "orange")
lines(seq(0, 20, by = .1), probsMeanFour, lwd = 3, lty = 4, col = "purple")
legend("topright", legend = c("mean = 1", "mean = 2", "mean = 4"),
     col = c("green", "orange", "purple"), lwd = 2)
```

Probability Distribution Function for Exponential Dists with Means = 1, 2



The reason to choose $by = 0.1$ instead of $by = 4$ is because the smaller the increment, the more well-defined our graph is going to be.

(2.2) (4 points) Following the example in class 8, get a random sample of 20 observations from an exponential distribution with mean 4. Repeat this process 10000 times. Save your results in a matrix called `samples` with 10,000 rows and 20 columns. Display the dimension of `samples`. Show the first 3 rows of `samples` but round the values to two decimal places.

```
# To make grading easier, please leave the following line of code in your assignment
set.seed(230)
```

```
# FILL IN REMAINING CODE
samples <- matrix(nrow = 10000, ncol = 20)
for (i in 1:nrow(samples)){
  samples[i, ] <- rexp(20, rate = 0.25)
}

dim(samples)
```

```
## [1] 10000    20
```

```
round(head(samples, 3), 2)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13] [,14]
## [1,] 5.29 9.12 1.03 9.29 5.59 0.13 0.41 1.71 4.10 3.40 1.66 0.42 3.56 0.15
## [2,] 1.80 1.28 0.51 5.28 7.92 0.82 0.17 2.97 8.35 14.77 0.11 0.53 2.76 5.12
## [3,] 2.40 4.94 3.63 2.16 0.26 2.49 7.93 1.20 9.42 7.67 2.75 3.70 1.27 1.99
##      [,15] [,16] [,17] [,18] [,19] [,20]
## [1,] 5.87 2.38 1.60 3.76 9.61 2.19
## [2,] 1.14 7.47 4.03 4.78 10.15 4.85
## [3,] 0.30 3.50 1.30 1.83 3.90 3.51
```

(2.3) (4 points) Calculate the sample mean for each sample of size 20 (i.e. calculate the mean for each row of `samples`). Repeat this process to get the sample median and the sample variance for each sample of size 20. Save these values in objects called, respectively, `smeans`, `smedians`, `svariance`.

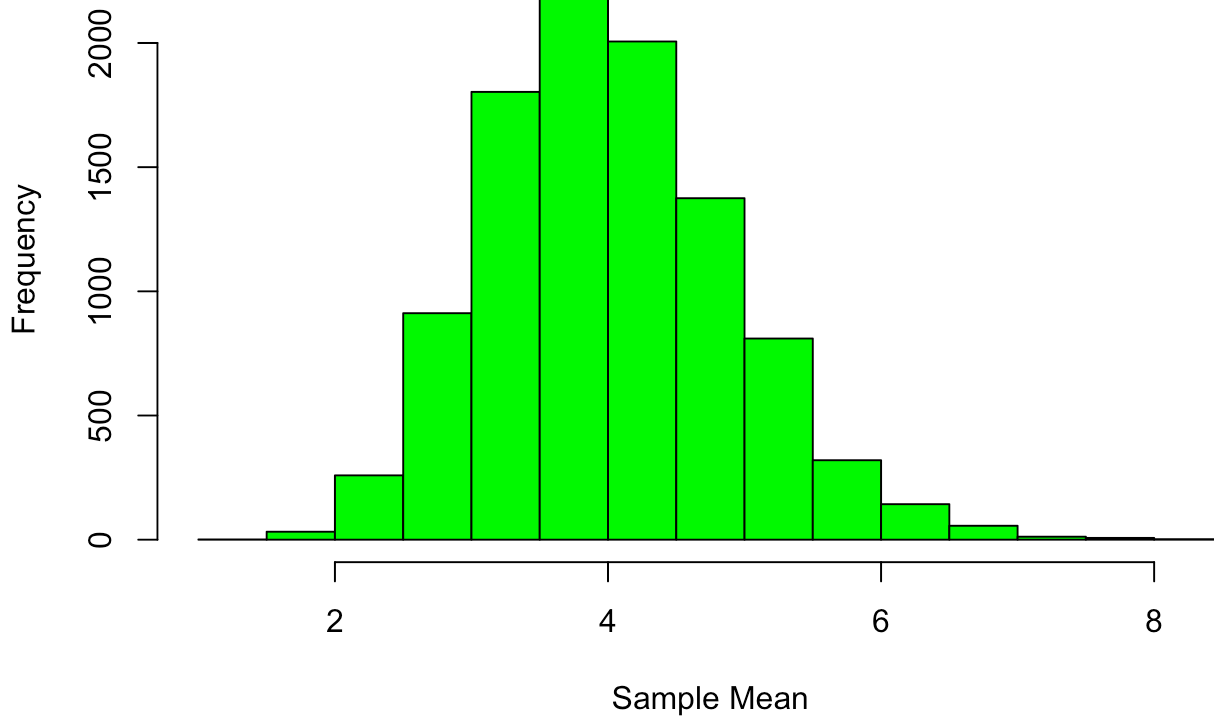
```
smeans <- apply(samples, 1, mean)
smedians <- apply(samples, 1, median)
svariance <- apply(samples, 1, var)
```

(2.4) (9 points)

- Create a sample histogram of the sample means (make the bars green, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample means using the `qqPlot()` function in the `car` package. Comment on whether the CLT seems to be in effect.
- Get summary statistics OF THE SAMPLE MEANS and save this to an object called `ans1`. Using code, display only the element of `ans1` that is the sample mean, rounded to two decimal places. Is this the value you expect?
- Calculate and display the sample standard deviation of the sample means (use the function `sd()`) and display rounded to two decimal places. Then, use code to calculate the value you'd expect based on the CLT, again rounded to two decimal places. Are the two values similar?

```
hist(smeans,
     col = "green",
     main = "Bootstrapped Sample Means of an Exponential Distribution, Mean = 4",
     xlab = "Sample Mean",
     ylab = "Frequency")
```

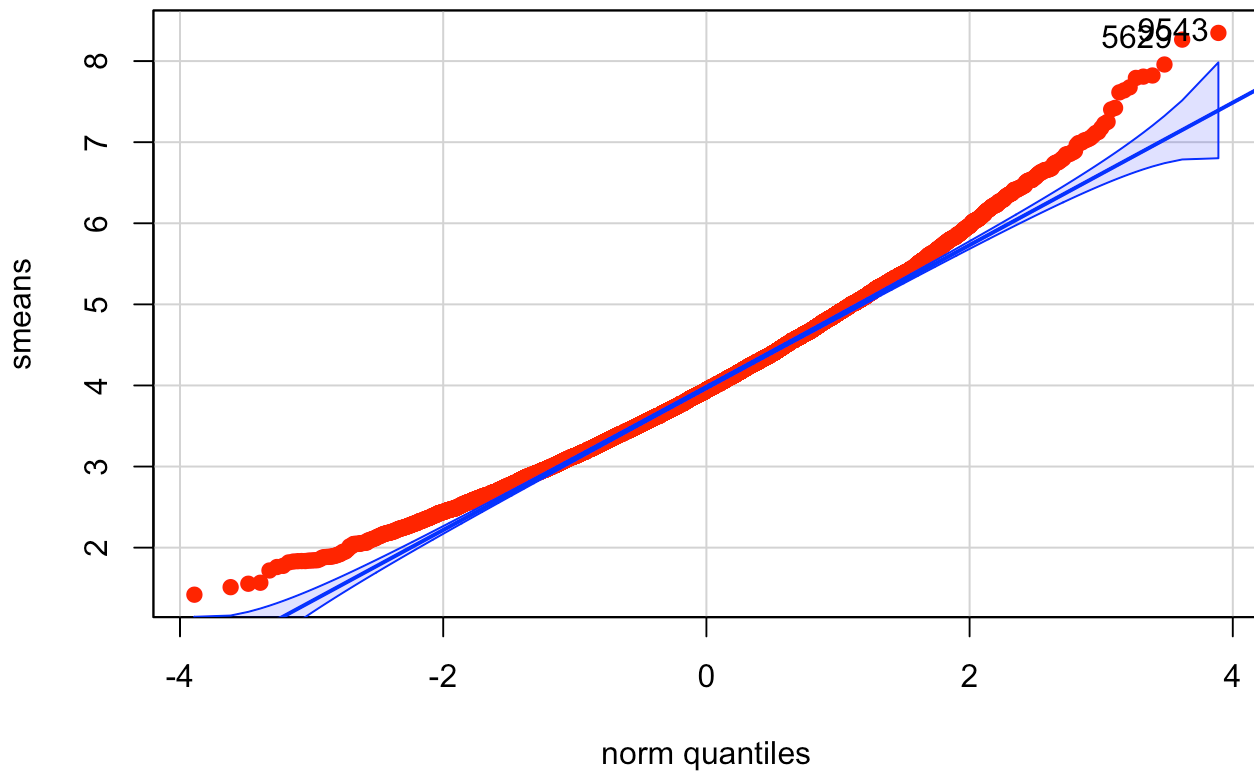
Bootstrapped Sample Means of an Exponential Distribution, Mean = 4



```
library(car)
```

```
## Loading required package: carData
```

```
qqPlot(smeans, col = "red", pch = 19)
```



```
## [1] 9543 5629
```

```
ans1 <- summary(smeans)
round(ans1["Mean"], 2)
```

```
## Mean
## 4.01
```

```
round(sd(smeans), 2)
```

```
## [1] 0.89
```

```
round(4 / sqrt(20), 2)
```

```
## [1] 0.89
```

The sample means are not quite normally distributed (CLT not in effect). The mean of `ans1` being 4.01 is a value that falls within expectations given that the mean of our exponential distribution was 4. The sample and calculated standard deviations are the same (rounded).

(2.5) (7 points)

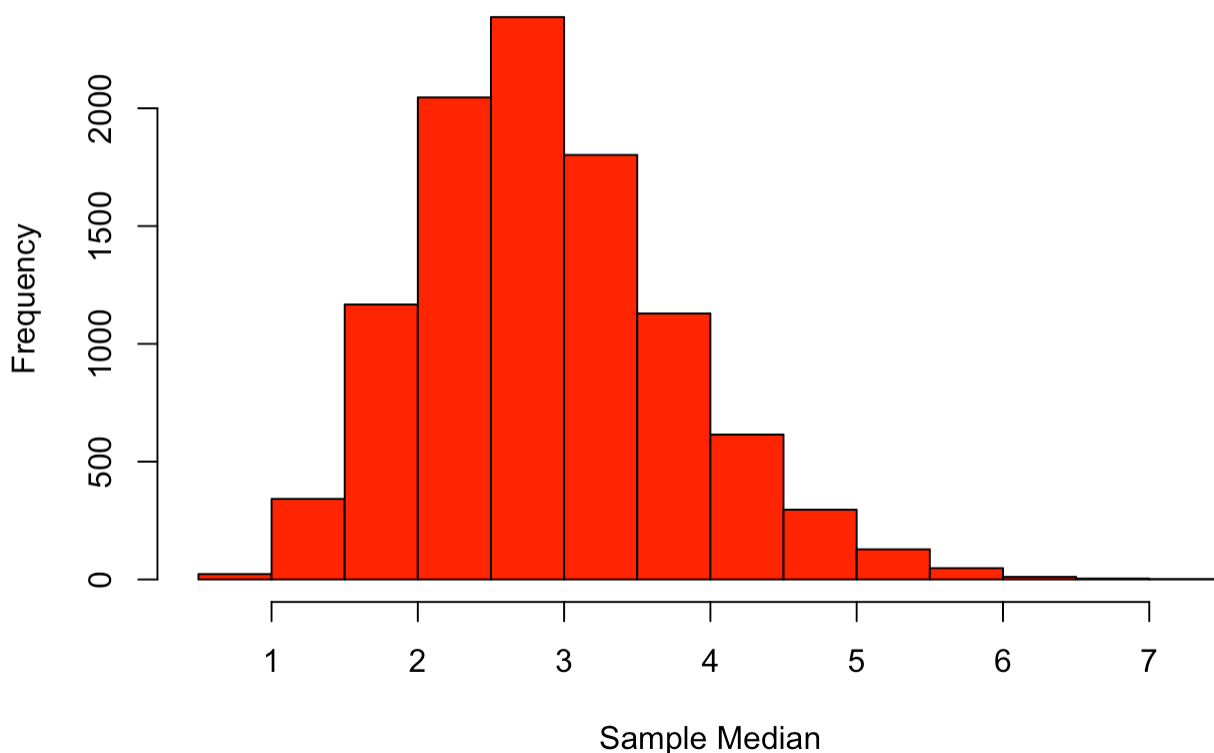
- Create a sample histogram of the sample MEDIANS (make the bars red, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample medians using the `qqPlot()` function in the `car` package. Do the medians

seem normally distributed?

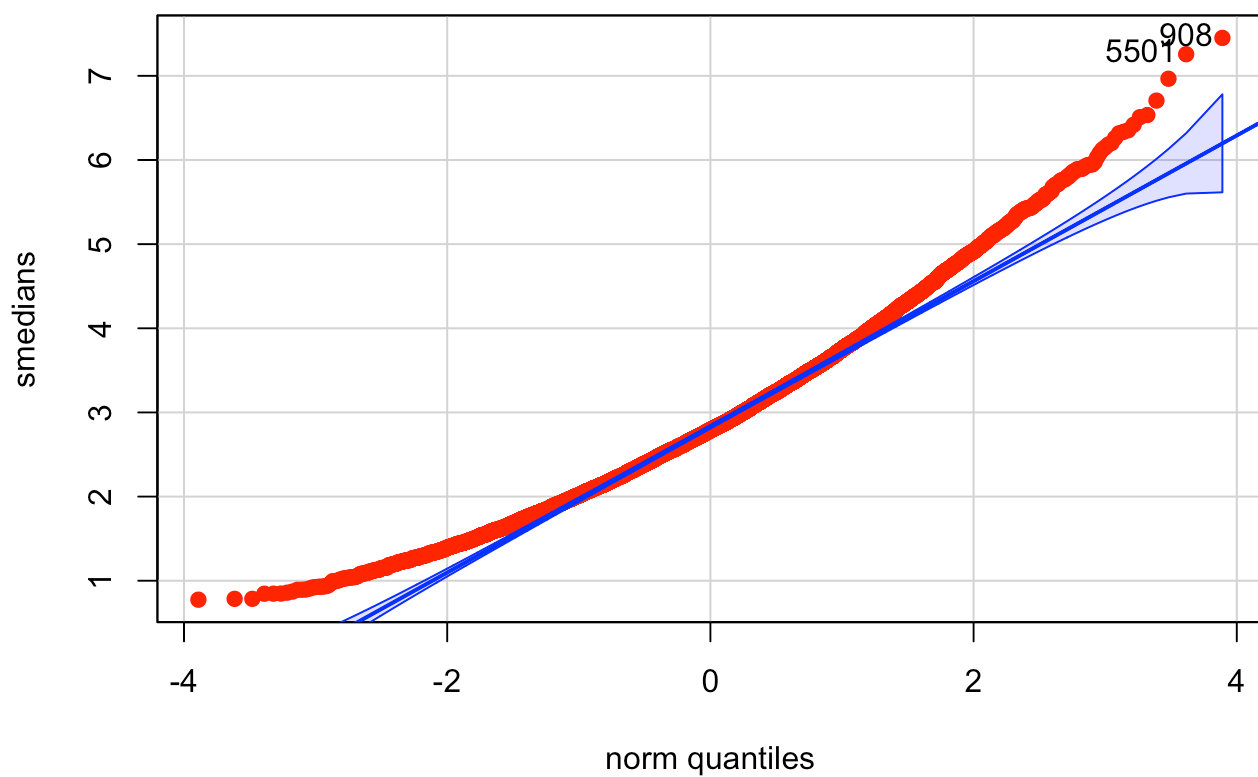
- Display summary statistics OF THE SAMPLE MEDIANS. Is the median of the sample medians the value you expect?
- Calculate and display the sample standard deviation of the sample medians and display rounded to two decimal places. Is this value similar to the value you would expect? *Note that (despite what AI may tell you - I was given an incorrect answer twice), the standard deviation of the sample median is the same as the standard deviation of the sample mean.*

```
hist(smedians,  
     col = "red",  
     main = "Bootstrapped Sample Medians of an Exponential Distribution, Mean = 4",  
     xlab = "Sample Median",  
     ylab = "Frequency")
```

Bootstrapped Sample Medians of an Exponential Distribution, Mean = 4



```
qqPlot(smedians, col = "red", pch = 19)
```



```
## [1] 908 5501
```

```
ans2 <- summary(smedians)
ans2
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7747  2.2452   2.7884   2.8817  3.4133   7.4508
```

```
round(ans2["Median"], 2)
```

```
## Median
##  2.79
```

```
median_value <- qexp(0.5, rate = 0.25)
round(median_value, 2)
```

```
## [1] 2.77
```

```
round(sd(smedians), 2)
```

```
## [1] 0.88
```

```
round(4 / sqrt(20), 2)
```

```
## [1] 0.89
```

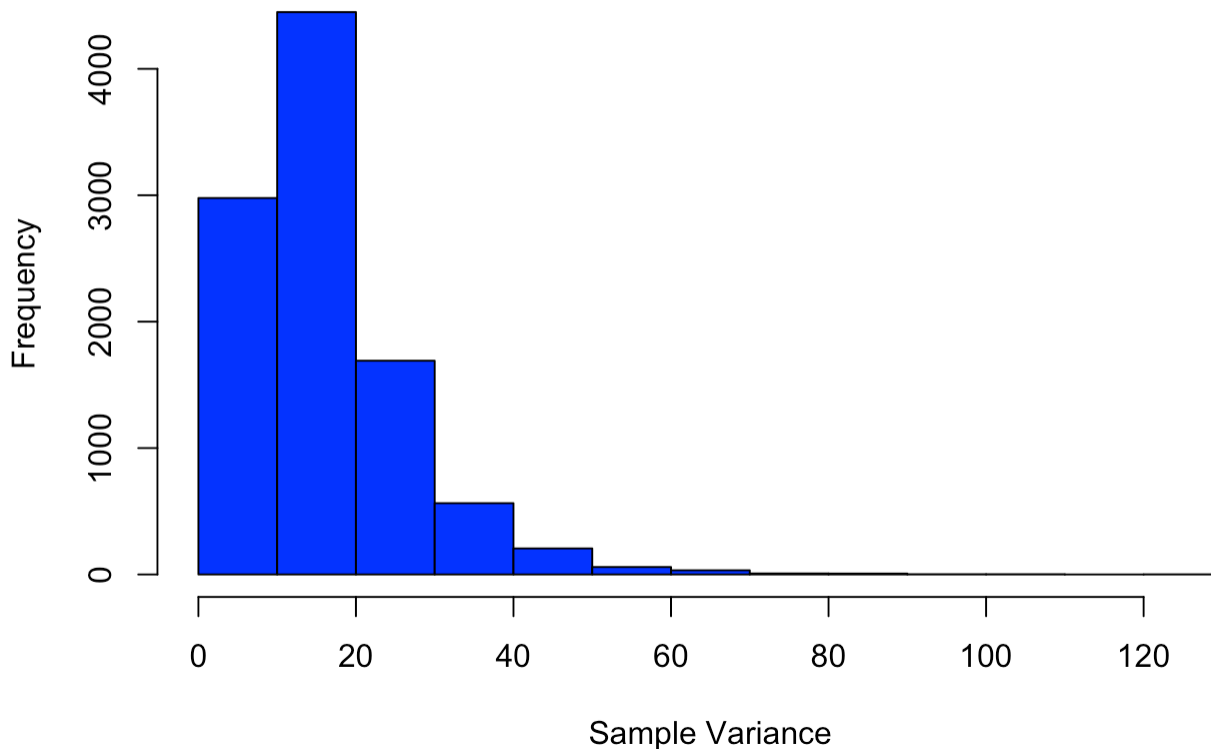
Based on the normal quantile plot, the sample medians are not quite normally distributed (CLT not in effect). The median of the sample values is close to what I'd expect, since the median of an exponential distribution with mean = 4 is expected to be 2.77 and our sample median is 2.79. The sample sd of the sample medians is similar to the value that I'd expect since the sample sd is 0.88 and we'd expect an sd of 0.89.

(2.6) (6 points)

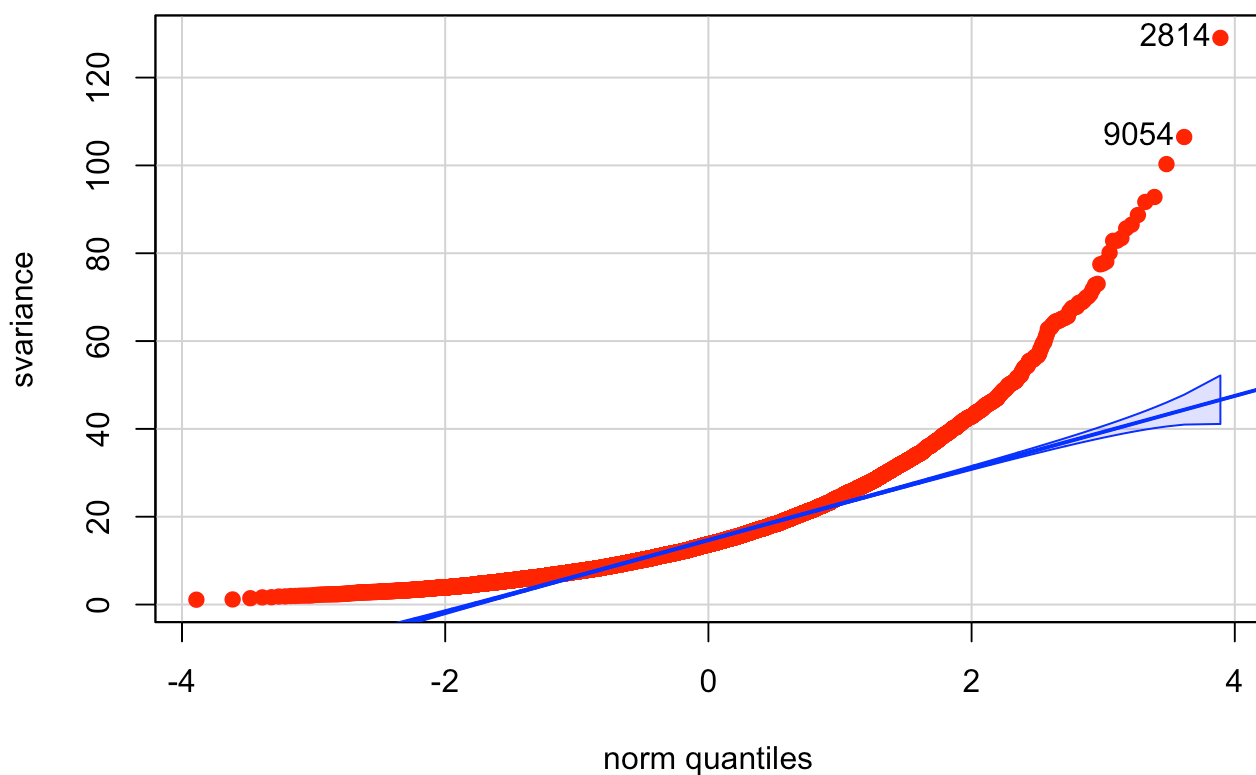
- Create a sample histogram of the sample VARIANCES (make the bars blue, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample VARIANCES using the `qqPlot()` function in the `car` package. Do the variances seem normally distributed?
- Display summary statistics OF THE SAMPLE VARIANCES. Is the mean of the sample variances the value you expect?

```
hist(svariance,
     col = "blue",
     main = "Bootstrapped Sample Variances of an Exponential Distribution, Mean = 4",
     xlab = "Sample Variance",
     ylab = "Frequency")
```

Bootstrapped Sample Variances of an Exponential Distribution, Mean =



```
qqPlot(svariance, col = "red", pch = 19)
```



```
## [1] 2814 9054
```

```
ans3 <- summary(svariance)
ans3
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  1.108   9.149  13.697  16.087  20.230 129.035
```

```
round(ans3["Mean"], 2)
```

```
## Mean
## 16.09
```

The variance does not seem normally distributed, especially on the right side The mean of sample variances is within a range I'd expect, as the expected variance is 16 and our sample variance is 16.09

(3) Iron Concentration and the Bootstrap (50 points, 5 points each part, parts 3.5 and 3.8 count double.

This problem examines results of a study looking at iron concentration in the the water runoff of watersheds with different mining histories above different rock types. The data is [HERE \(https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/IronMineRock.csv\)](https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/IronMineRock.csv). The variables are Rock , Mine , Iron (concentration) and logIron which is the natural log of iron concentration.

(3.1) Read the data into an object called `iron` . Show the first 6 rows of the data. Show the unique values of `Rock` and `Mine` . Then modify `iron` so that it only contains data for Unmined locations. How many observations remain? How many remaining observations do you have for each rock type? Incidentally, the rock types are Limestone and Sandstone.

```
iron <- read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/IronMineRock.csv")
head(iron, 6)
```

```
##   Rock   Mine Iron logIron
## 1 Sand Unmined 0.20  -1.61
## 2 Sand Unmined 0.25  -1.39
## 3 Sand Unmined 0.04  -3.22
## 4 Sand Unmined 0.06  -2.81
## 5 Sand Unmined 1.20   0.18
## 6 Sand Unmined 0.30  -1.20
```

```
unique(iron$Rock)
```

```
## [1] "Sand" "Lime"
```

```
unique(iron$Mine)
```

```
## [1] "Unmined" "Reclaimed" "Abandoned"
```

```
iron <- iron[iron$Mine == "Unmined", ]
sum(!is.na(iron$Iron))
```

```
## [1] 26
```

```
sum(!is.na(iron$logIron))
```

```
## [1] 26
```

```
table(iron$Rock[!is.na(iron$Iron)])
```

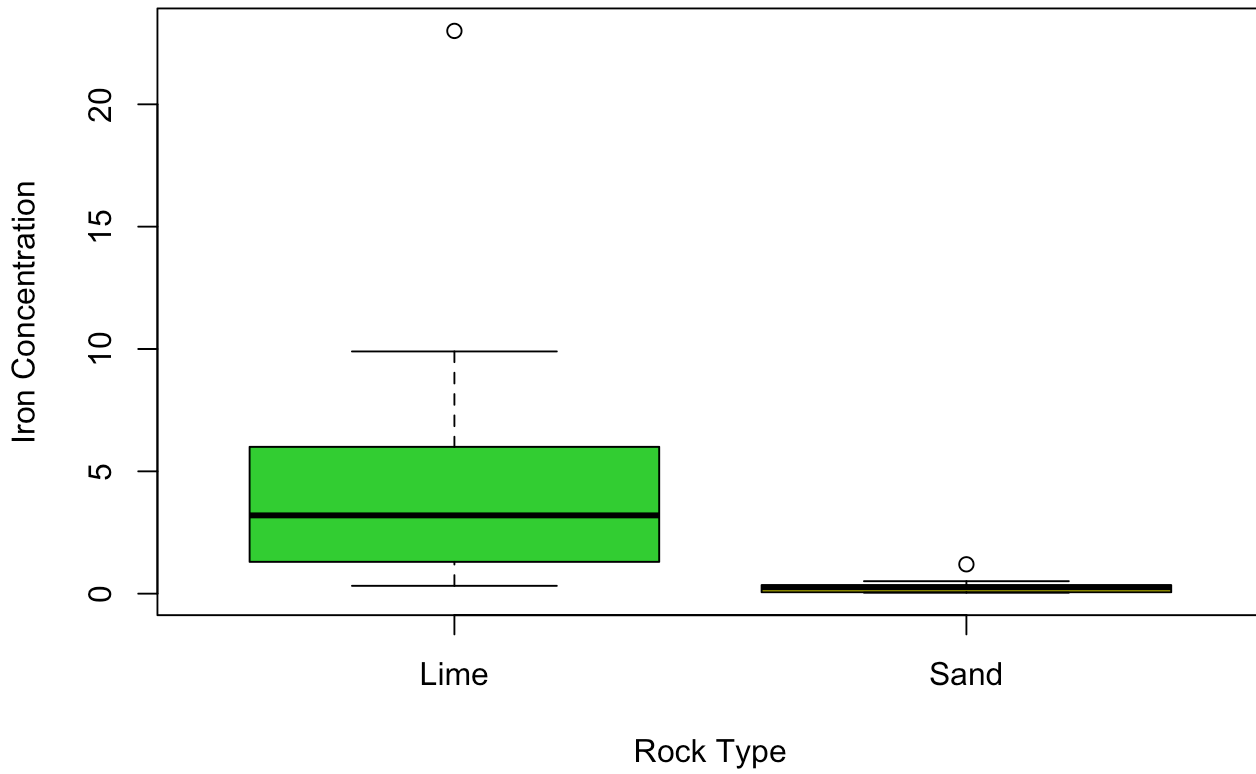
```
##
## Lime Sand
##   13   13
```

After modifying 'iron' to only contain rows for unmined locations, 26 observations remain. Of these 26, 13 are 'Lime' and 13 are 'Sand'

(3.2) Make side by side boxplots of iron concentration by rock type. Make this same plot for the natural log of the iron concentration. Write a sentence or two about what you observe. Which scale do you prefer?

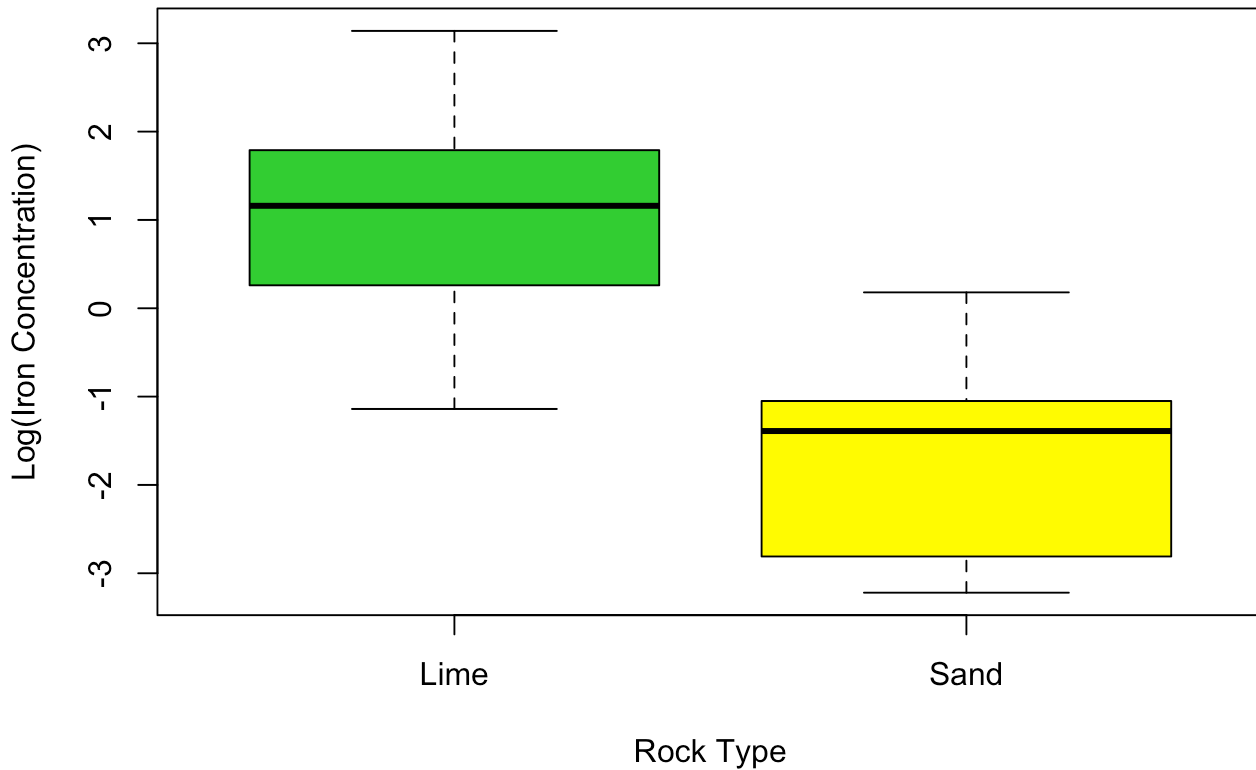
```
boxplot(iron$Iron ~ iron$Rock,
        col = c("limegreen", "yellow"),
        main = "Iron Concentration by Rock Type",
        xlab = "Rock Type",
        ylab = "Iron Concentration")
```

Iron Concentration by Rock Type



```
boxplot(iron$logIron ~ iron$Rock,  
        col = c("limegreen", "yellow"),  
        main = "Log(Iron Concentration) by Rock Type",  
        xlab = "Rock Type",  
        ylab = "Log(Iron Concentration)")
```

Log(Iron Concentration) by Rock Type



I prefer the log scale because it is a lot less squished and we can actually see what's happening for both Lime and Sand

(3.3) Calculate summary statistics for iron concentration by rock type on the raw scale and the log scale.

```
tapply(iron$Iron, iron$Rock, summary)
```

```
## $Lime
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.32   1.30   3.20   5.17   6.00   23.00
##
## $Sand
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.0400 0.0600 0.2500 0.2985 0.3500 1.2000
```

```
tapply(iron$logIron, iron$Rock, summary)
```

```
## $Lime
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -1.140   0.260   1.160   1.032   1.790   3.140
##
## $Sand
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -3.220  -2.810  -1.390  -1.677  -1.050   0.180
```

(3.4) Calculate a two-sample t-test comparing mean log iron concentration between rock types. Save results in an object called `test1` and display the results. Use $\alpha = .01$ (i.e. make a 99% CI). Is there evidence of a difference between rock types?

Create an object called 'test2' that has similar results for the raw iron concentration. Is there evidence of a difference between rock types?

```
test1 <- t.test(iron$logIron ~ iron$Rock, conf.level = 0.99)
test2 <- t.test(iron$Iron ~ iron$Rock, conf.level = 0.99)
test1
```

```
##
##  Welch Two Sample t-test
##
## data:  iron$logIron by iron$Rock
## t = 5.9803, df = 23.524, p-value = 3.88e-06
## alternative hypothesis: true difference in means between group Lime and group Sand is not equal to 0
## 99 percent confidence interval:
##  1.439543 3.977381
## sample estimates:
## mean in group Lime mean in group Sand
##           1.031538           -1.676923
```

```
test2
```

```
##
##  Welch Two Sample t-test
##
## data:  iron$Iron by iron$Rock
## t = 2.873, df = 12.062, p-value = 0.01395
## alternative hypothesis: true difference in means between group Lime and group Sand is not equal to 0
## 99 percent confidence interval:
##  -0.3029436 10.0460206
## sample estimates:
## mean in group Lime mean in group Sand
##           5.1700000           0.2984615
```

In the logIron t-test, there is evidence of difference between rock types because our p-value is less than 0.01 (3.88e-06) and our 99 percent confidence interval does not contain 0. In the raw iron concentration t-test, there is NO evidence of difference between rock types because our p-value is above 0.01 (0.01395), and our 99 percent confidence interval contains 0

(3.5) Get 10,000 bootstrap samples from the data and calculate the difference the in mean log iron concentration between rock types for each bootstrap sample. Save these means in an object called `diffLogIron` . In your code, use variable names that make sense.

```
# To make grading easier, please leave the following line of code in your assignment
set.seed(230)
```

```
# FILL IN REMAINING CODE
```

```
N = 10000
```

```
diffLogIron <- rep(NA, N)
```

```
for (i in 1:N){
```

```
  sLime <- sample(iron$logIron[iron$Rock == "Lime"], sum(iron$Rock == "Lime"), replace = TRUE)
```

```
  sSand <- sample(iron$logIron[iron$Rock == "Sand"], sum(iron$Rock == "Sand"), replace = TRUE)
```

```
  diffLogIron[i] <- mean(sLime) - mean(sSand)
```

```
}
```

(3.6) Calculate a 99% Bootstrap confidence interval. Show your results rounded to two decimal places.

```
ci <- quantile(diffLogIron, c(0.005, 0.995))
```

```
round(ci, 2)
```

```
## 0.5% 99.5%
```

```
## 1.60 3.82
```

(3.7) Make a histogram of bootstrap differences in means and add vertical lines for the theoretical and bootstrapped confidence intervals (and also add a legend explaining which lines are which type of confidence interval). Make a normal quantile plot of the bootstrapped differences. Discuss what you observe in both plots.

```
hist(diffLogIron,
      col = "blue",
      main = "Bootstrapped Sample Means Diff in Log(Iron Concentration)",
      xlab = "Difference in Log(Iron Concentration) Means",
      ylab = "Frequency")
```

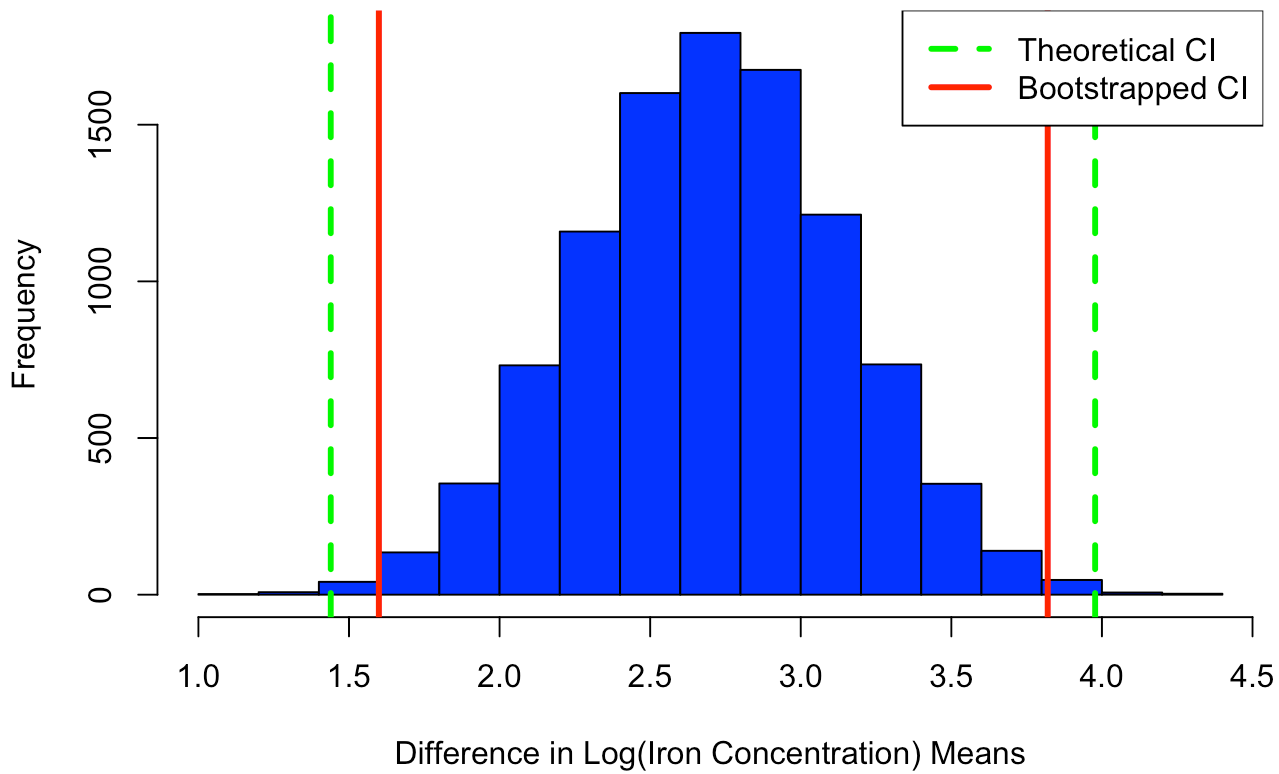
```
#Add lines to histogram for CI's
```

```
abline(v = ci, lwd = 3, col = "red")
```

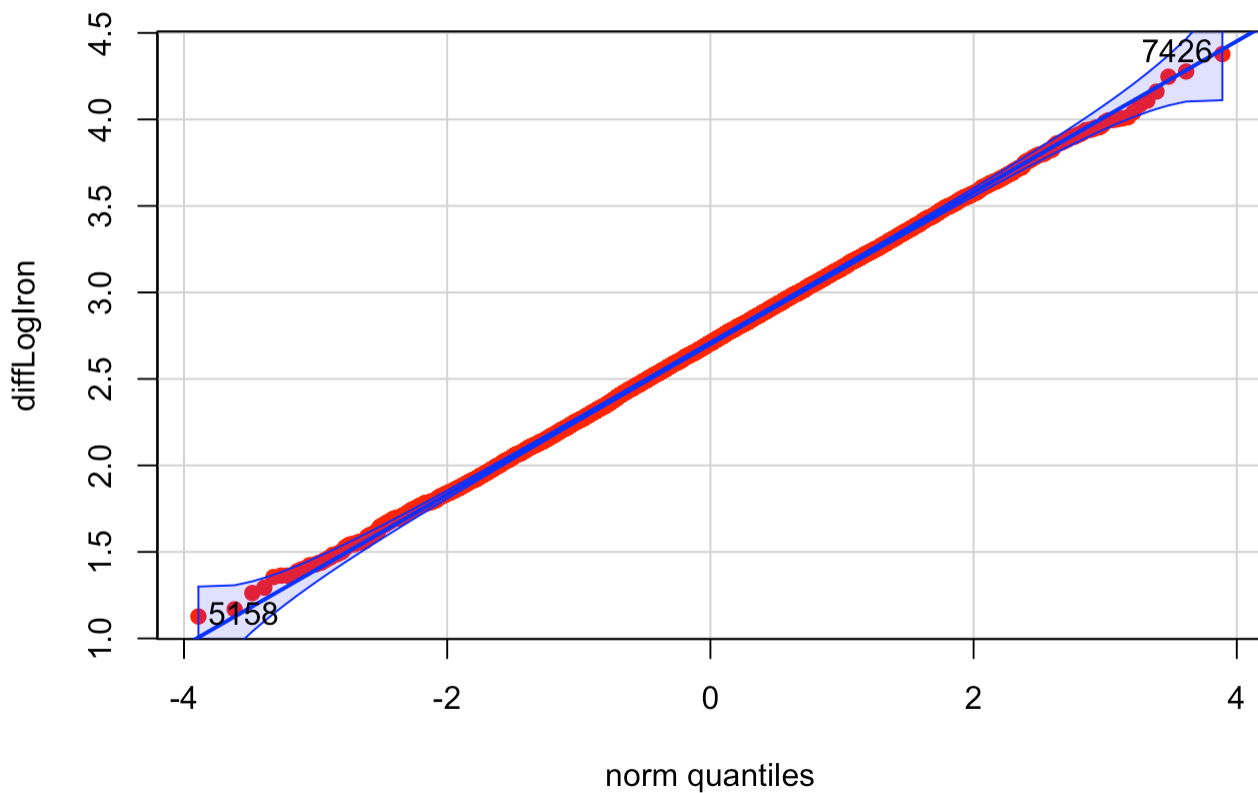
```
abline(v = test1$conf.int, lwd = 3, col = "green", lty = 2)
```

```
legend("topright", c("Theoretical CI", "Bootstrapped CI"), lwd = 3, col = c("green", "red"), lty = c(2, 1))
```

Bootstrapped Sample Means Diff in Log(Iron Concentration)



```
qqPlot(diffLogIron, col = "red", pch = 19)
```



```
## [1] 7426 5158
```

In the histogram, we see that diffLogIron follows a normal distribution. We also see that the bootstrapped CI is slightly narrower than the theoretical CI, but both CIs seem to be centered around the same point. In the qqplot, we can pretty clearly see that diffLogIron follows a normal distribution the whole way through.

3.8) Finally, repeat 3.5 through 3.7 for iron concentrations on the raw scale. Discuss your results. Note that you'll need to add the option `xlim = c(-1, 12)` to your histogram.

```
# To make grading easier, please leave the following line of code in your assignment
set.seed(230)

# FILL IN REMAINING CODE
N = 10000
diffRawIron <- rep(NA, N)
for (i in 1:N){
  sLime <- sample(iron$Iron[iron$Rock == "Lime"], sum(iron$Rock == "Lime"), replace = TRUE)
  sSand <- sample(iron$Iron[iron$Rock == "Sand"], sum(iron$Rock == "Sand"), replace = TRUE)
  diffRawIron[i] <- mean(sLime) - mean(sSand)
}

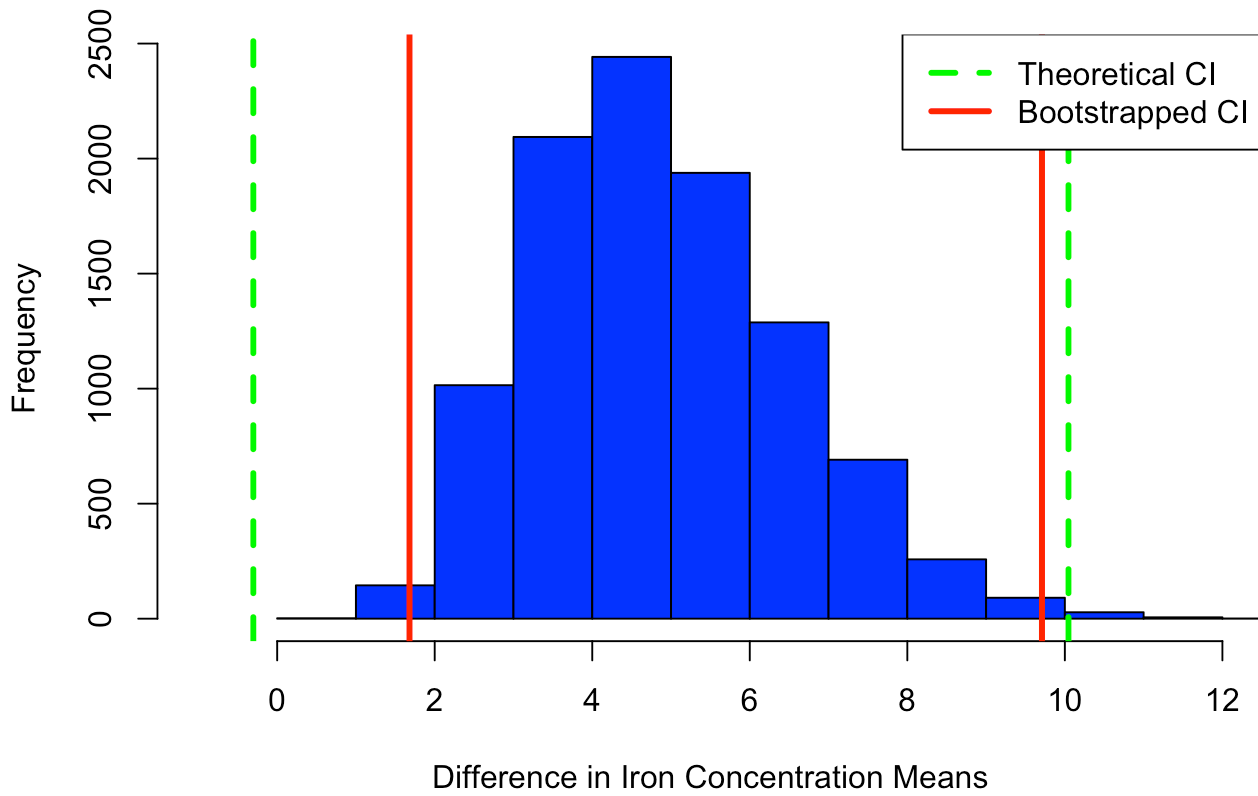
ci <- quantile(diffRawIron, c(0.005, 0.995))
round(ci, 2)
```

```
## 0.5% 99.5%
## 1.68 9.71
```

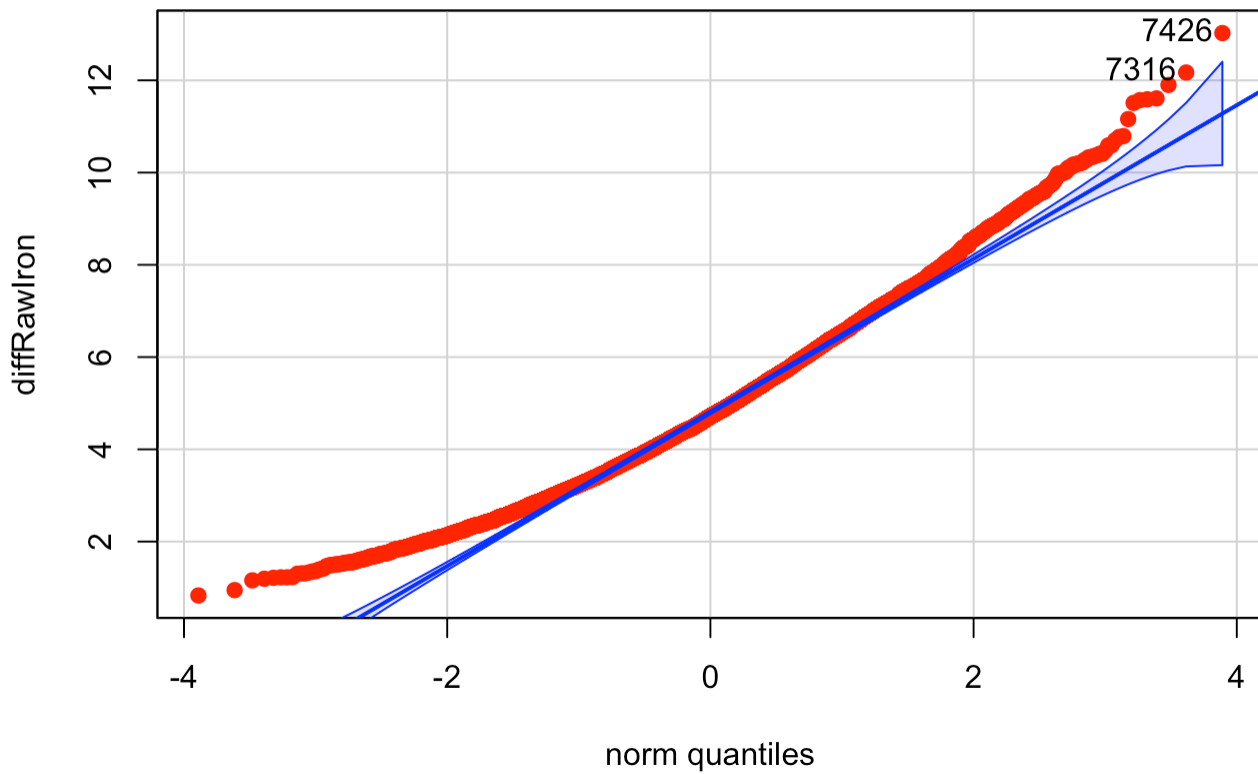
```
hist(diffRawIron,
      col = "blue",
      main = "Bootstrapped Sample Means Diff in Iron Concentration",
      xlab = "Difference in Iron Concentration Means",
      ylab = "Frequency",
      xlim = c(-1, 12))

#Add lines to histogram for CI's
abline(v = ci, lwd = 3, col = "red")
abline(v = test2$conf.int, lwd = 3, col = "green", lty = 2)
legend("topright", c("Theoretical CI", "Bootstrapped CI"), lwd = 3, col = c("green", "red"), lty =
c(2,1))
```

Bootstrapped Sample Means Diff in Iron Concentration



```
qqPlot(diffRawIron, col = "red", pch = 19)
```



Looking at the histogram, we can see that the data is somewhat right skewed and that the theoretical and bootstrapped confidence intervals differ both in range and where it is centered. The qqplot reflects the skewed nature of the histogram, as it doesn't seem to follow a normal distribution that well. This shows that the log transformation of the iron concentration data was able to identify a better pattern than the raw values.

4. Pseudo-coding

You are analyzing long-term water quality data from the Connecticut River. The dataset includes solute concentrations of major ions (Calcium, Magnesium, Sodium, Potassium, Chloride, Nitrate, Phosphate, and Bicarbonate) measured at locations across New England in New Hampshire, Vermont, Massachusetts, and Connecticut. Measurements have been collected from the 1950s to the present. Each row of the dataset includes the state, date, discharge, and solute name & concentration (so there are separate rows for different ions on the same date).

Write pseudocode describing the steps you would take to compare concentrations across states, examine long-term temporal trends by decade, investigate concentration–discharge (C–Q) relationships (literally concentration vs. discharge), and test whether there are statistically significant differences between groups using the tools you have learned so far.

- *Read in the data using read.csv()*
- *Use head() to get an idea of how the data is formatted*
- *Check how many empty cells there are and if we need to do some data preprocessing (imputing, removing, etc.)*
- *To compare concentrations across states*
 - *Make boxplots for each solute by state (so we would have a graph for Calcium concentration by state)*
 - *If this boxplot is messy/unreadable, then do some transformations (such as natural log)*
 - *Use t-tests to test for statistically significant differences between groups -We would do this for pairs of states at a time*
 - *We can also bootstrap if we needed more data*
- *To examine long-term temporal trends by decade*
 - *Group the data by decade (can do this by creating a new “decade” column)*
 - *Bootstrap the data*
 - *Make a line plot using the means of the bootstrap data*
 - *Use t-tests between decades to test for significant difference*
 - *Calculate CI from the bootstrap to test for significant difference*
- *To investigate C-Q relationships*
 - *Bootstrap the data if we don't have sufficient data*
 - *Create a scatterplot*
 - *Calculate the CI of our data to see if there's statistically significant difference*